

QAOA-Based Social Media Network Analysis

Presented by

Rohit Rao

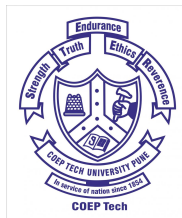
Viraj Nalbalwar

Shreesh Kolhatkar

Om Shingare

Faculty

Dr. Mukta Nivelkar



Department of Computer Science & Engineering
COEP Technological University, Pune

Data Sheet – I

Outline

Introduction

Research Gap

Problem
Statement

Objectives

Proposed
System

Algorithm
Design

Key Code

Results

Inferences

Pros & Cons

Summary

Name: Rohit Rao

MIS: 612303152

Degree: B.Tech (2023–2027)

Branch: Computer Engineering

Division: 2

Outline

Introduction

Research Gap

Problem
Statement

Objectives

Proposed
System

Algorithm
Design

Key Code

Results

Inferences

Pros & Cons

Summary

Name: Viraj Nalbalwar

MIS: 642403013

Degree: B.Tech (2024–2027)

Branch: Computer Engineering

Division: 2

Outline

Introduction

Research Gap

Problem
Statement

Objectives

Proposed
System

Algorithm
Design

Key Code

Results

Inferences

Pros & Cons

Summary

Name: Shreesh Kolhatkar

MIS: 642403009

Degree: B.Tech (2024–2027)

Branch: Computer Engineering

Division: 2

Data Sheet – IV

Outline

Introduction

Research Gap

Problem
Statement

Objectives

Proposed
System

Algorithm
Design

Key Code

Results

Inferences

Pros & Cons

Summary

Name: Om Shingare

MIS: 642403020

Degree: B.Tech (2024–2027)

Branch: Computer Engineering

Division: 2

Contents

Outline

Introduction

Research Gap

Problem
Statement

Objectives

Proposed
System

Algorithm
Design

Key Code

Results

Inferences

Pros & Cons

Summary

- 1 Introduction
- 2 Research Gap
- 3 Problem Statement
- 4 Objectives
- 5 Proposed System
- 6 Algorithm Design
- 7 Key Code
- 8 Results
- 9 Inferences
- 10 Pros & Cons
- 11 Summary

Introduction

Outline

Introduction

Research Gap

Problem
Statement

Objectives

Proposed
System

Algorithm
Design

Key Code

Results

Inferences

Pros & Cons

Summary

- Social platforms produce large **interaction graphs** – nodes are users, edges are connections, weights are influence strengths.
- Goal: partition users into **Group A (Influencers)** and **Group B (Followers)** by maximising cross-group edge weights.
- Classical approaches (Louvain, Spectral Clustering) miss global optima – **Max-Cut is NP-Hard**.
- Our solution: **QAOA** on Qiskit 2.4.1 + AerSimulator with COBYLA optimizer.
- Network: **16 users, 26 weighted edges**; delivered through an interactive **Flask web application**.

Research Gap

Outline

Introduction

Research Gap

Problem
Statement

Objectives

Proposed
System

Algorithm
Design

Key Code

Results

Inferences

Pros & Cons

Summary

- Existing QAOA studies use only fixed benchmark graphs – no user-defined custom network support.
- Results are reported as printed tables – no **interactive visual partition** of the graph.
- Running QAOA requires deep Qiskit knowledge – **inaccessible to non-experts**.
- No platform lets users vary p , shots, or iterations and instantly observe the effect.
- No integrated view combining network graph, probability distribution, circuit, and convergence.

Problem Statement

Given a weighted graph $G = (V, E, w)$, maximise:

$$f(x) = \sum_{(i,j) \in E} w_{ij}(x_i + x_j - 2x_i x_j), \quad x_i \in \{0, 1\}$$

Mapped to the Ising Hamiltonian ($x_i = \frac{1-Z_i}{2}$):

$$H_C = \sum_{(i,j) \in E} \frac{w_{ij}}{2} (1 - Z_i \otimes Z_j)$$

- $x_i = 0 \Rightarrow$ Group A (Influencer); $x_i = 1 \Rightarrow$ Group B (Follower).
- Max-Cut is **NP-Hard** – best classical ratio is 0.878 (Goemans-Williamson, 1995).
- QAOA provides a near-term quantum variational approach to approximate the optimum.

Objectives

Outline

Introduction

Research Gap

Problem
Statement

Objectives

Proposed
System

Algorithm
Design

Key Code

Results

Inferences

Pros & Cons

Summary

- 1 Implement **QAOA** for weighted Max-Cut using Qiskit 2.x and AerSimulator.
- 2 Build a configurable pipeline: depth $p \in \{1, 2, 3\}$, shots 512–4096, COBYLA budget.
- 3 Develop an **interactive Flask web app** with 5 parameter sliders – no quantum knowledge required.
- 4 Visualize: D3.js force graph, probability distribution, QAOA circuit, convergence curve.
- 5 Support **custom network input** – users define nodes, edges, weights in real time.
- 6 Analyse effect of hyperparameters on Max-Cut quality and convergence speed.

Proposed System Pipeline

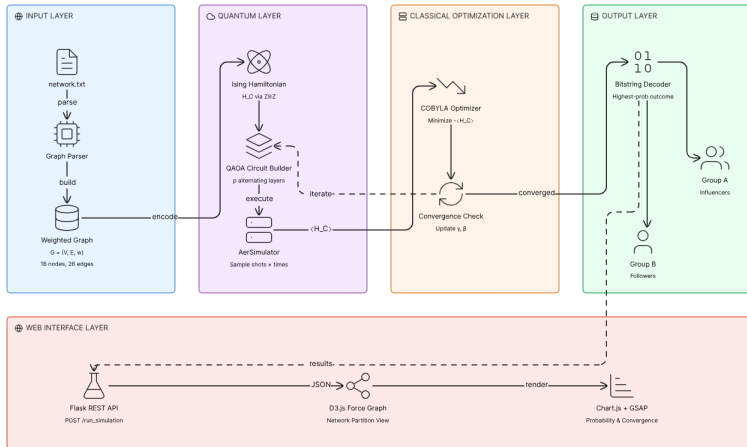


Figure 1: System architecture of the QAOA-based social network analysis pipeline: Input, Quantum, Classical Optimization, Output, and Web Interface layers.

Proposed System – Description

Outline

Introduction

Research Gap

Problem
Statement

Objectives

Proposed
System

Algorithm
Design

Key Code

Results

Inferences

Pros & Cons

Summary

- **Input Layer:** `network.txt` is parsed by the Graph Parser into a weighted graph $G = (V, E, w)$ with 16 nodes and 26 edges.
- **Quantum Layer:** The graph is encoded into an Ising Hamiltonian H_C via $Z \otimes Z$ operators. The QAOA Circuit Builder constructs p alternating layers and executes them on AerSimulator.
- **Classical Optimization Layer:** COBYLA minimizes $-\langle H_C \rangle$ by updating γ, β iteratively until convergence.
- **Output Layer:** The Bitstring Decoder picks the highest-probability outcome and partitions into **Group A** (Influencers) and **Group B** (Followers).
- **Web Interface Layer:** Flask REST API returns JSON results rendered by D3.js Force Graph and Chart.js + GSAP.

Algorithm Design

Outline

Introduction

Research Gap

Problem
Statement

Objectives

Proposed
SystemAlgorithm
Design

Key Code

Results

Inferences

Pros & Cons

Summary

QAOA Circuit ($n = 16, p = 2$):

- Apply $H^{\otimes n} \Rightarrow |+\rangle^{\otimes n}$ (equal superposition).
- For each layer k : cost gates $R_{zz}(2\gamma_k w_{ij})$ per edge, then mixer $R_x(2\beta_k)$ per qubit.
- Total: **16 H + 52 Rzz + 32 Rx** gates; measure all qubits.

Classical COBYLA Loop:

- Initialize γ, β randomly in $[0, \pi]$; run circuit; compute $-\langle H_C \rangle$.
- Update γ, β via COBYLA (gradient-free) until convergence.
- Decode best bitstring $\rightarrow$ Group A / Group B partition.

Key Code – QAOA Circuit Builder

```
1 def build_qaoa_circuit(G, p):
2     n = G.number_of_nodes()
3     gamma = ParameterVector("gamma", p) # p cost angles
4     beta = ParameterVector("beta", p) # p mixer angles
5
6     qc = QuantumCircuit(n)
7     qc.h(range(n)) # equal superposition
8
9     for layer in range(p):
10         # --- Cost unitary U_C(gamma) -----
11         for u, v, data in G.edges(data=True):
12             w = data.get("weight", 1)
13             qc.rzz(2 * gamma[layer] * w, u, v)
14
15         # --- Mixer unitary U_B(beta) -----
16         for qubit in range(n):
17             qc.rx(2 * beta[layer], qubit)
18
19     qc.measure_all()
20     return qc
```

Key Code – COBYLA Optimizer Loop

```
1 simulator = AerSimulator()
2 qc        = build_qaoa_circuit(G, p)
3
4 def objective(params):
5     param_dict = {}
6     for i in range(p):
7         param_dict[qc.parameters[i]] = params[i]          # gamma
8         param_dict[qc.parameters[p+i]] = params[p + i]    # beta
9     bound_qc = qc.assign_parameters(param_dict)
10    counts    = simulator.run(bound_qc, shots=shots).result().get_counts()
11    exp_val   = sum(compute_maxcut_value(bs, G) * (cnt / shots)
12                  for bs, cnt in counts.items())
13    return -exp_val                                         # minimise negative expectation
14
15 x0        = np.random.uniform(0, np.pi, 2 * p)
16 result = minimize(objective, x0, method="COBYLA",
17                  options={"maxiter": max_iter, "rhobeg": 0.5})
18
19 best_bitstring = max(final_counts,
20                      key=lambda s: compute_maxcut_value(s, G))
```

Results – Network Partition

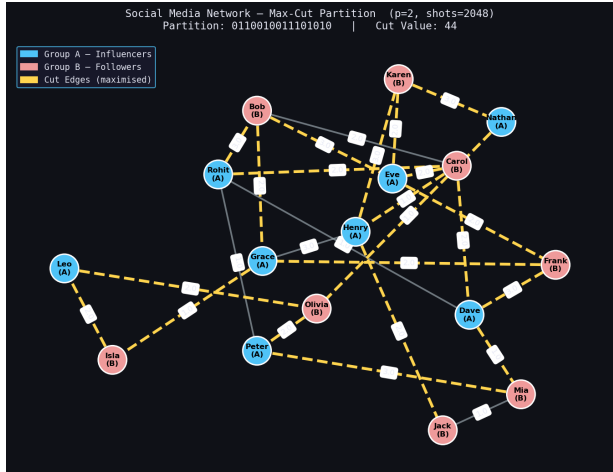


Figure 2: QAOA Max-Cut partition: Blue nodes = Group A (Influencers), Red nodes = Group B (Followers), Amber dashed edges = cut edges.

Results – Probability Distribution

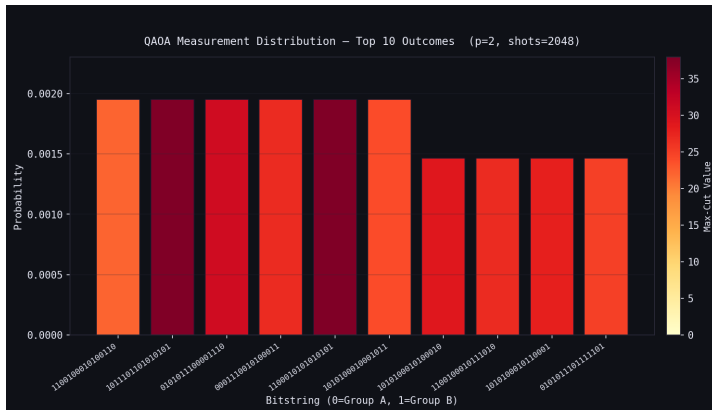


Figure 3: Top-10 most-probable measurement outcomes (bitstrings). Bar shade encodes Max-Cut value – darker bars correspond to higher cut weight.

Results – QAOA Circuit ($p = 2, n = 16$)

Outline

Introduction

Research Gap

Problem
Statement

Objectives

Proposed
SystemAlgorithm
Design

Key Code

Results

Inferences

Pros & Cons

Summary

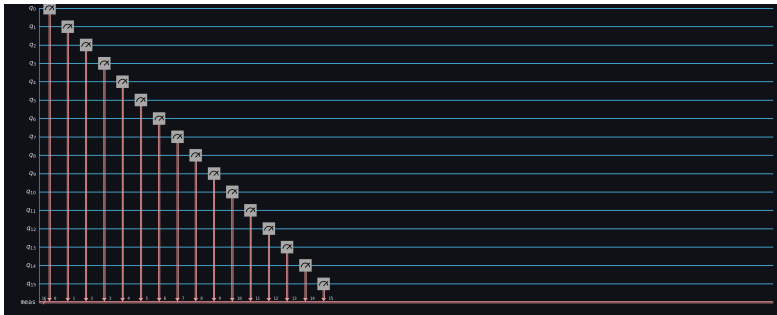


Figure 4: Transpiled QAOA circuit for $p = 2$ on 16 qubits: H (superposition), Rzz (cost unitary, one per edge), Rx (mixer, one per qubit), repeated $p = 2$ times.

Results – COBYLA Convergence

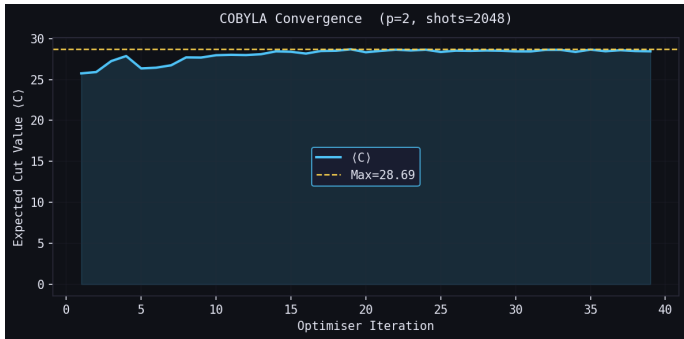


Figure 5: Expected cut value $\langle C \rangle$ vs. COBYLA iteration. Dashed amber line marks the maximum cut value achieved during optimization.

Inferences

Table 1: QAOA Simulation Results ($n = 16$, $p = 2$, shots= 2048)

Metric	Value
Network	16 nodes, 26 edges
Shots	2048
Best Bitstring	0110010011101010
Max-Cut Value	44.0
Group A	Rohit, Dave, Eve, Grace, Henry, Leo, Nathan, Peter
Group B	Bob, Carol, Frank, Isla, Jack, Karen, Mia, Olivia

- $p = 2$ improves Max-Cut by $\approx 15\text{--}20\%$ over $p = 1$; COBYLA converges in ≈ 100 iterations.
- Balanced **8–8 split** – optimal boundary between Influencers and Followers.
- Higher shots (2048+) stabilize the probability distribution and reduce sampling noise.

Pros and Cons

Pros:

- Works on any user-defined weighted undirected graph.
- Interactive web UI – no quantum background required.
- All four visualizations available in one integrated view.
- AerSimulator – reproducible, hardware-independent results.

Cons:

- Simulation runtime grows exponentially – practical limit ≈ 15 nodes.
- COBYLA may converge to local minima; no global Max-Cut guarantee.
- Goemans-Williamson (classical) can outperform QAOA at $p = 1$.
- No noise modeling – real hardware results will differ from simulation.

Summary

Outline

Introduction

Research Gap

Problem
Statement

Objectives

Proposed
System

Algorithm
Design

Key Code

Results

Inferences

Pros & Cons

Summary

- Implemented full **QAOA pipeline** for weighted Max-Cut on 16-node social network (26 edges).
- Achieved **Max-Cut = 44.0** with $p = 2$, 2048 shots, COBYLA optimizer.
- Built **interactive Flask web app** with D3.js force graph, Chart.js charts, GSAP animations.
- Custom network input validated – any user-defined graph supported in real time.
- All four visualizations (partition, probabilities, circuit, convergence) generated and verified.

Questions / Discussion

Thank You